Gramática del analizador sintáctico de Lua 5.5

Martin Moreno Jara

Notación

Los no terminales se escriben así: ⟨NON_TERMINAL⟩ y los terminales en minúsculas y negrilla. El símbolo inicial de la gramática es es ⟨CHUNK⟩. ε es la cadena vacía, $\rightarrow$ introduce una regla y | separa sus alternativas. La gramática es LL(1).

1. Estructura del programa

Símbolo inicial, bloques y la sentencia `return`.

⟨CHUNK⟩ → ⟨BLOCK⟩

⟨BLOCK⟩ → ⟨STATLIST⟩ ⟨RETSTAT⟩

⟨STATLIST⟩ → ⟨STAT⟩ ⟨STATLIST⟩ | ε

⟨RETSTAT⟩ → **`return`** ⟨MAYBE_EXPLIST⟩ ⟨OPT_SEMICOLON⟩ | ε

⟨OPT_SEMICOLON⟩ → `';'` | ε

2. Sentencias

Las asignaciones y las llamadas se factorizan después del primer nombre.

⟨STAT⟩ → `';'`
| ⟨NAME⟩ ⟨VAR_STAT⟩
| ⟨LABEL⟩
| **`break`**
| **`goto`** ⟨NAME⟩
| **`while`** ⟨EXP⟩ ⟨DO_BLOCK_END⟩
| **`repeat`** ⟨BLOCK⟩ **`until`** ⟨EXP⟩
| ⟨IFSTAT⟩
| ⟨FORSTAT⟩
| **`function`** ⟨FUNCNAME⟩ ⟨FUNCBODY⟩
| ⟨LOCAL_DEC⟩
| ⟨GLOBAL_DEC⟩

⟨VAR_STAT⟩ → `'['` ⟨EXP⟩ `']'` ⟨VAR_STAT⟩
| `'.'` ⟨NAME⟩ ⟨VAR_STAT⟩
| ⟨FUNCTIONCALL⟩ ⟨FUNCALL_STAT⟩
| ⟨MORE_VAR⟩ `'='` ⟨EXPLIST⟩

⟨FUNCALL_STAT⟩ → `'['` ⟨EXP⟩ `']'` ⟨VAR_STAT⟩

$$
\begin{aligned}
&\quad | \texttt{'.'}\ \langle \text{NAME} \rangle\ \langle \text{VAR_STAT} \rangle \\
&\quad | \langle \text{FUNCTIONCALL} \rangle\ \langle \text{FUNCALL_STAT} \rangle \ \mid\ \varepsilon \\
\langle \text{LABEL} \rangle &\to \texttt{'::'}\ \langle \text{NAME} \rangle\ \texttt{'::'} \\
\langle \text{DO_BLOCK_END} \rangle &\to \textbf{do}\ \langle \text{BLOCK} \rangle\ \textbf{end} \\
\langle \text{IFSTAT} \rangle &\to \textbf{if}\ \langle \text{EXP} \rangle\ \textbf{then}\ \langle \text{BLOCK} \rangle\ \langle \text{ELSEIFSTAT} \rangle\ \langle \text{ELSESTAT} \rangle\ \textbf{end} \\
\langle \text{ELSEIFSTAT} \rangle &\to \textbf{elseif}\ \langle \text{EXP} \rangle\ \textbf{then}\ \langle \text{BLOCK} \rangle\ \langle \text{ELSEIFSTAT} \rangle \ \mid\ \varepsilon \\
\langle \text{ELSESTAT} \rangle &\to \textbf{else}\ \langle \text{BLOCK} \rangle \ \mid\ \varepsilon \\
\langle \text{FORSTAT} \rangle &\to \textbf{for}\ \langle \text{NAME} \rangle\ \langle \text{FORTYPE} \rangle\ \langle \text{DO_BLOCK_END} \rangle \\
\langle \text{FORTYPE} \rangle &\to \langle \text{NUMERICAL_FOR} \rangle \mid \langle \text{GENERIC_FOR} \rangle \\
\langle \text{NUMERICAL_FOR} \rangle &\to \texttt{'='}\ \langle \text{EXP} \rangle\ \texttt{','}\ \langle \text{EXP} \rangle\ \langle \text{OPT_EXP} \rangle \\
\langle \text{OPT_EXP} \rangle &\to \texttt{','}\ \langle \text{EXP} \rangle \ \mid\ \varepsilon \\
\langle \text{GENERIC_FOR} \rangle &\to \langle \text{MORE_NAMES} \rangle\ \textbf{in}\ \langle \text{EXPLIST} \rangle
\end{aligned}
$$

3. Declaraciones

`local` y `global`.

$$
\begin{aligned}
\langle \text{LOCAL_DEC} \rangle &\to \texttt{local}\ \langle \text{LOCAL_OPTS} \rangle \\
\langle \text{LOCAL_OPTS} \rangle &\to \langle \text{FUNC_DEC} \rangle \mid \langle \text{ATTNAME_BRANCH} \rangle \\
\langle \text{ATTNAME_BRANCH} \rangle &\to \langle \text{ATTNAMELIST} \rangle\ \langle \text{OPT_EXPLIST} \rangle \\
\langle \text{GLOBAL_DEC} \rangle &\to \texttt{global}\ \langle \text{GLOBAL_OPTS} \rangle \\
\langle \text{GLOBAL_OPTS} \rangle &\to \langle \text{FUNC_DEC} \rangle \mid \langle \text{ATTNAMELIST} \rangle\ \langle \text{OPT_EXPLIST} \rangle \\
\langle \text{FUNC_DEC} \rangle &\to \texttt{function}\ \langle \text{NAME} \rangle\ \langle \text{FUNCBODY} \rangle \\
\langle \text{OPT_EXPLIST} \rangle &\to \texttt{'='}\ \langle \text{EXPLIST} \rangle \ \mid\ \varepsilon \\
\langle \text{ATTNAMELIST} \rangle &\to \langle \text{NAME} \rangle\ \langle \text{ATTRIB} \rangle\ \langle \text{NAME_ATTRIB_LIST} \rangle \\
\langle \text{NAME_ATTRIB_LIST} \rangle &\to \texttt{','}\ \langle \text{NAME} \rangle\ \langle \text{ATTRIB} \rangle\ \langle \text{NAME_ATTRIB_LIST} \rangle \ \mid\ \varepsilon \\
\langle \text{ATTRIB} \rangle &\to \texttt{'<'}\ \langle \text{NAME} \rangle\ \texttt{'>'} \ \mid\ \varepsilon
\end{aligned}
$$

4. Funciones

Nombres de función, cuerpos y listas de parámetros.

$$
\begin{aligned}
\langle \text{FUNCNAME} \rangle &\to \langle \text{NAME} \rangle\ \langle \text{DOTTED_FUNCNAME} \rangle\ \langle \text{COLON_FUNCNAME} \rangle \\
\langle \text{DOTTED_FUNCNAME} \rangle &\to \texttt{'.'}\ \langle \text{NAME} \rangle\ \langle \text{DOTTED_FUNCNAME} \rangle \ \mid\ \varepsilon
\end{aligned}
$$

⟨COLON_FUNCNAME⟩ → `':'` ⟨NAME⟩ | ε

⟨FUNCTIONDEF⟩ → **`function`** ⟨FUNCBODY⟩

⟨FUNCBODY⟩ → `'('` ⟨OPT_PARLIST⟩ `')'` ⟨BLOCK⟩ **`end`**

⟨OPT_PARLIST⟩ → ⟨PARLIST⟩ | ε

⟨PARLIST⟩ → ⟨NAME⟩ ⟨PARLIST_TAIL⟩ | ⟨VARPARAGRAM⟩

⟨PARLIST_TAIL⟩ → `','` ⟨PARLIST_AFTER_COMMA⟩ | ε

⟨PARLIST_AFTER_COMMA⟩ → ⟨NAME⟩ ⟨PARLIST_TAIL⟩ | ⟨VARPARAGRAM⟩

⟨VARPARAGRAM⟩ → `'...'` ⟨OPT_NAME⟩

⟨OPT_NAME⟩ → ⟨NAME⟩ | ε

5. Listas

Listas de variables, de nombres y de expresiones.

⟨MORE_NAMES⟩ → `','` ⟨NAME⟩ ⟨MORE_NAMES⟩ | ε

⟨MORE_VAR⟩ → `','` ⟨PREFIXEXP⟩ ⟨MORE_VAR⟩ | ε

⟨MAYBE_EXPLIST⟩ → ⟨EXPLIST⟩ | ε

⟨EXPLIST⟩ → ⟨EXPLIST_ITEM⟩ ⟨MORE_EXP⟩

⟨MORE_EXP⟩ → `','` ⟨EXPLIST_ITEM⟩ ⟨MORE_EXP⟩ | ε

⟨EXPLIST_ITEM⟩ → ⟨EXP⟩ | ⟨FUNCTIONDEF⟩

6. Expresiones

Operandos, expresiones prefijo y llamadas.

⟨EXP⟩ → ⟨OPERAND⟩ ⟨EXP_PRIME⟩

⟨EXP_PRIME⟩ → ⟨ARITH_TAIL⟩ ⟨CMP_TAIL⟩ ⟨LOGIC_TAIL⟩

⟨OPERAND⟩ → ⟨NAME⟩ ⟨PREFIXEXP_PRIME⟩ | ⟨OPERAND_NOT_ID⟩

⟨OPERAND_NOT_ID⟩ → **`nil`** | **`false`** | **`true`**
| **`valor_numerico`** | **`cadena_de_caracteres`** | ⟨TABLECONSTRUCTOR⟩
| ⟨UNOP⟩ ⟨OPERAND⟩ | `'('` ⟨EXP⟩ `')'`

⟨PREFIXEXP⟩ → ⟨NAME⟩ ⟨PREFIXEXP_PRIME⟩

⟨PREFIXEXP_PRIME⟩ → `'['` ⟨EXP⟩ `']'` ⟨PREFIXEXP_PRIME⟩
| `'.'` ⟨NAME⟩ ⟨PREFIXEXP_PRIME⟩
| ⟨FUNCTIONCALL⟩ ⟨PREFIXEXP_PRIME⟩ | ε

⟨FUNCTIONCALL⟩ → ⟨ARGS⟩ | `':'` ⟨NAME⟩ ⟨ARGS⟩

⟨ARGS⟩ → `'('` ⟨MAYBE_EXPLIST⟩ `')'`

7. Tablas

Constructores de tablas.

⟨TABLECONSTRUCTOR⟩ → `'{'` ⟨FIELDLIST⟩ `'}'`

⟨FIELDLIST⟩ → ⟨FIELD⟩ ⟨FIELD_TAIL⟩ | ε

⟨FIELD_TAIL⟩ → ⟨FIELDSEP⟩ ⟨FIELDLIST⟩ | ε

⟨FIELD⟩ → `'['` ⟨EXP⟩ `']'` `'='` ⟨FIELD_VALUE⟩
| ⟨NAME⟩ ⟨FIELD_ID⟩
| ⟨OPERAND_NOT_ID⟩ ⟨EXP_PRIME⟩
| ⟨FUNCTIONDEF⟩

⟨FIELD_ID⟩ → `'='` ⟨FIELD_VALUE⟩
| ⟨PREFIXEXP_PRIME⟩ ⟨EXP_PRIME⟩

⟨FIELD_VALUE⟩ → ⟨EXP⟩ | ⟨FUNCTIONDEF⟩

⟨FIELDSEP⟩ → `','` | `';'`

8. Operadores

Operadores binarios por nivel de precedencia, operadores unarios y nombres (identificadores).

⟨LOGIC_TAIL⟩ → ⟨LOGICAL_OP⟩ ⟨CMP_EXP⟩ ⟨LOGIC_TAIL⟩ | ε

⟨CMP_EXP⟩ → ⟨ARTH_EXP⟩ ⟨CMP_TAIL⟩

⟨CMP_TAIL⟩ → ⟨RELATIONAL_OP⟩ ⟨ARTH_EXP⟩ | ε

⟨ARTH_EXP⟩ → ⟨OPERAND⟩ ⟨ARITH_TAIL⟩

⟨ARITH_TAIL⟩ → ⟨FLAT_OP⟩ ⟨OPERAND⟩ ⟨ARITH_TAIL⟩ | ε

⟨FLAT_OP⟩ → ⟨ARITH_OP⟩ | ⟨BITWISE_OP⟩ | ⟨OTHER_OP⟩

⟨ARITH_OP⟩ → `'+'` | `'-'` | `'*'` | `'/'`
| `'//'` | `'^'` | `'%'`

⟨RELATIONAL_OP⟩ → `'<'` | `'<='` | `'>'`
| `'>='` | `'=='` | `'~='`

⟨LOGICAL_OP⟩ → **`and`** | **`or`**

⟨BITWISE_OP⟩ → `'&'` | `'~'` | `'|'`
| `'>>'` | `'<<'`

⟨OTHER_OP⟩ → `'..'`

⟨UNOP⟩ → `'-'` | **`not`** | `'#'`

⟨NAME⟩ → **`id`** | **`print`** | **`pcall`**
| **`error`** | **`warn`** | **`xpcall`**

9. Correspondencia entre símbolos y tokens

Símbolo que se escribe en la gramática y el tipo de token `tkn_...` que le corresponde tanto en el analizador léxico como en el sintáctico.

`'&'`	`tkn_bit_and`	`'|'`	`tkn_bit_or`
`'~'`	`tkn_bitex_or`	`'>>'`	`tkn_right_shift`
`'<<'`	`tkn_left_shift`	`';'`	`tkn_semicolon`
`':'`	`tkn_colon`	`','`	`tkn_comma`
`'.'`	`tkn_period`	`'::'`	`tkn_goto`
`'..'`	`tkn_concat`	`'...'`	`tkn_varargs`
`'{'`	`tkn_opening_key`	`'}'`	`tkn_closing_key`
`'['`	`tkn_opening_bra`	`']'`	`tkn_closing_bra`
`'('`	`tkn_opening_par`	`')'`	`tkn_closing_par`
`'#'`	`tkn_length`	`'+'`	`tkn_plus`
`'-'`	`tkn_minus`	`'*'`	`tkn_times`
`'/'`	`tkn_div`	`'//'`	`tkn_floor_div`
`'^'`	`tkn_power`	`'%'`	`tkn_mod`
`'=='`	`tkn_equal`	`'~='`	`tkn_neq`
`'<='`	`tkn_leq`	`'>='`	`tkn_geq`
`'>'`	`tkn_greater`	`'<'`	`tkn_less`
`'='`	`tkn_assign`	**`valor_numerico`**	`tkn_num`
`cadena_de_caracteres`	`tkn_str`		